



WIA1002

DATA STRUCTURE

SEMESTER 2, 2025/2026

REPORT

SMART FOOD DELIVERY SYSTEM

LECTURER	DR. NARSIMLU KEMSARAM
OCCURRENCE	9

GROUP NAME: 6ix7even

GROUP MEMBERS:

NAME	MATRIC NUMBER
LAI JOE MING	23121171
LIM JIE XIN	25070088
HO HANN YI	23119400
NG YOO ZHENG	23121203
LIM YI-EN	23119574

Introduction.....	2
Background	3
Project Objectives	3
System Overview	3
System Approach & Design Architecture	4
System Approach.....	5
Architectural Flow	5
Module & Data Structure Integration.....	5
System Component Diagram	6
Logical System Data Flow	6
Data Structures & Algorithms	7
Comparative Summary of System Structures	12
Testing & Evaluation	13
1. Testing Methodology.....	14
3. Performance & Complexity Evaluation	15
Conclusion	16
1. Project Summary and Objective Fulfillment.....	17
2. Algorithmic Synthesis and System Efficiency	17
3. Limitations and Future Work	17

Introduction

Background

In the logistics and services industry, efficiency, speed, and reliability are key differentiators. For food delivery services, these qualities rely on the backend management of data. GoodTech requires a modern, scalable solution to manage its users, restaurants, orders, and delivery services.

We are designed and implemented a Smart Food Delivery & Order Management System using suitable data structures that enable fast data storage, retrieval, processing, and path optimization. The platform utilizes specialized, in-memory data structures. These structures model real-world behaviors and satisfy GoodTech's operational requirements.

To demonstrate the system's integration, we developed a Java-based application featuring both a text-based command-line interface (CLI) for backend simulation (Main.java) and a highly interactive, animated Graphical User Interface (GUI) dashboard (OnePageDashboard.java). This prototype proves how combining the proposed data structures creates an optimized, real-time logistics solution.

Project Objectives

To address GoodTech's operational challenges, this project establishes the following goals:

1. Develop an Integrated System: Combine six data structure solutions into a unified Java application.
2. Optimize Resource Allocation: Use Dijkstra's algorithm to automate delivery assignments and route planning.
3. Build an Interactive Interface: Create a real-time animated GUI dashboard showing live rider telemetry .
4. Evaluate Efficiency: Document the complexities of each data structure to justify their suitability for GoodTech's platform.

System Overview

GoodTech's operational scale exposes three major categories of system bottlenecks. First, managing accounts and order pipelines requires flexible storage as static structures suffer from expensive reallocations ($O(N)$) during customer and restaurant registration, while real-time order processing demands strict FIFO queueing combined with a LIFO stack to allow the undo capability before confirmation.

Second, the system needs to assign delivery riders efficiently based on priority, such as the nearest distance, using a method that performs better than a simple sequential

search. The system must also represent locations and implement an algorithm to determine the shortest path/route between the restaurant and the customer.

Third, the system must support fast searching and sorting of food items so that users can quickly locate items and view menus in an organized way. Finally, the system must provide fast, optimized access to important data like user profiles and order details to avoid latency.

System Approach & Design Architecture

System Approach

The Smart Food Delivery & Order Management System was developed using a modular, Object-Oriented Programming (OOP) approach in Java. Because the system handles multiple complex operations, a monolithic architecture was avoided using the Separation of Concerns principle. The core logic is divided into five isolated subsystems, where each module manages its own specialized Data Structure. To demonstrate versatility, these backend modules are integrated into a Dual-Interface System, accessible via both a Command-Line Interface (CLI) for backend simulation and a Graphical User Interface (GUI) for interactive visualization.

Architectural Flow

To ensure smooth integration, the system utilizes a "Hub-and-Spoke" architectural design with two primary entry points:

- **The CLI Hub (`Main.java` / `MainTerminal.java`):** Serves as the text-based router, capturing administrative inputs via the `Scanner` class.
- **The GUI Hub (`OnePageDashboard.java`):** Serves as the visual frontend, utilizing Java Swing to capture event-driven user actions (button clicks) and provide real-time animated telemetry (e.g., rider dispatching on a route map).
- **The Spokes (Sub-Managers):** Regardless of whether input comes from the CLI or GUI, execution is delegated to the core Manager classes (`UserRestaurantMenu`, `orderSystem`, `DeliveryManager`, `RouteManager`, and `SearchManager`). Each manager encapsulates its respective entity objects and directly manipulates its assigned Data Structures, ensuring structural isolation.

Module & Data Structure Integration

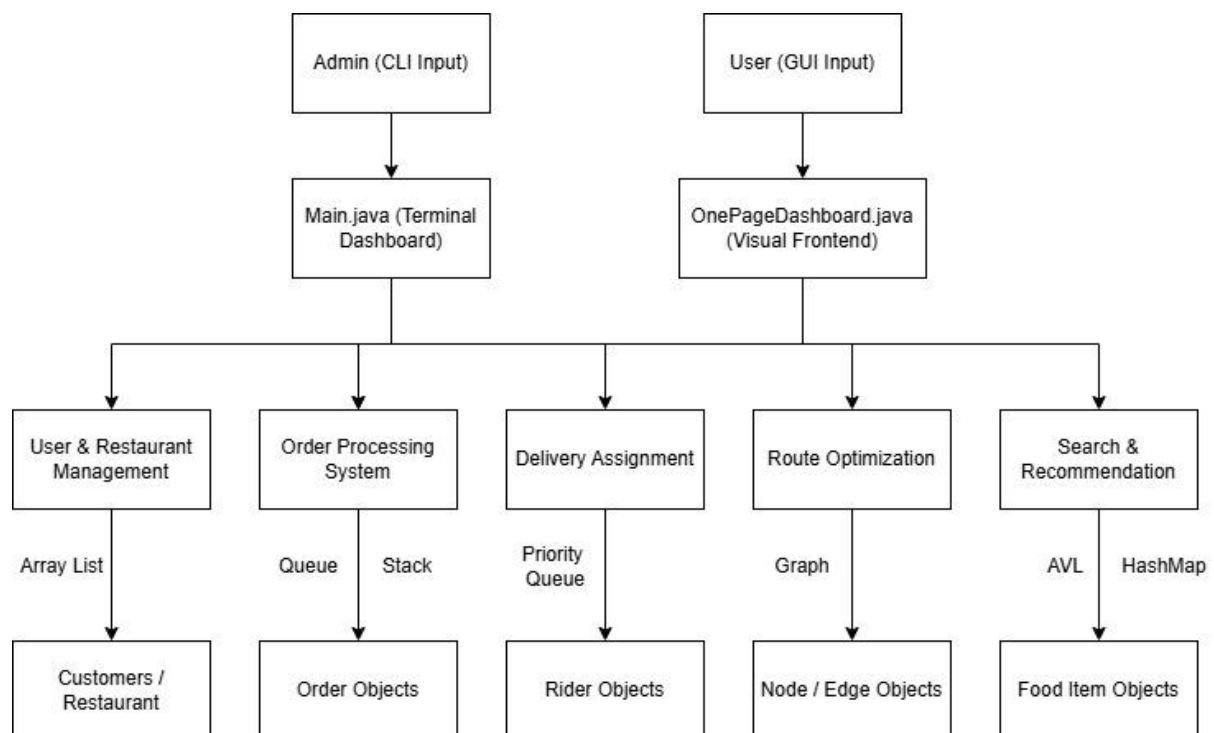
The central architecture connects the five core subsystems, utilizing specific data structures to achieve optimal Big-O time complexities for their respective tasks:

1. **User & Restaurant Management:** Utilizes **ArrayLists** for dynamic insertion, deletion, and iteration of customer and restaurant profiles.
2. **Real-Time Order Processing:** Employs a **Queue** (FIFO) to process incoming orders sequentially, and a **Stack** (LIFO) to provide an instant "Undo" feature for cart modifications.

3. **Delivery Assignment:** Utilizes a **Priority Queue (Min-Heap)** to prioritize and dispatch delivery riders based on the shortest delivery time or distance in $O(\log n)$ time.
4. **Route Optimization:** Implements a **Graph** data structure utilizing **Dijkstra's Algorithm** to calculate the absolute shortest path between restaurants and customers.
5. **Search & Data Retrieval:** Implements an **AVL Tree / BST** for $O(\log n)$ alphabetical sorting of food items, and a **HashMap** to achieve instant $O(1)$ retrieval of specific user and order records.

System Component Diagram

The following architectural diagram illustrates the integration of the Dual-Interface frontends with the five backend subsystems, highlighting the structural flow and the data structures assigned to each module.



Logical System Data Flow

While the modules are structurally isolated in the codebase, they interact sequentially to simulate a real-world delivery environment:

1. **Discovery:** A Customer retrieves their profile via the HashMap and searches for a Food Item using the AVL Tree.
2. **Transaction:** The Customer places an Order, which enters the FIFO Queue. Any modifications utilize the Stack's LIFO property.

3. **Dispatch:** The system queries the Delivery Min-Heap to extract the optimal Rider in $O(\log n)$ time.
4. **Routing & Animation:** The system calculates the shortest delivery route using the Graph data structure and Dijkstra's algorithm. If utilizing the GUI, this pathing data is fed back to the OnePageDashboard to animate the rider's physical movement across the screen.

Data Structures & Algorithms

Module 1: User and Restaurant Management System

Data Structure Selection: ArrayList (Java Collections Framework)

1. Suitability Justification

For handling user registrations and restaurant profiles, a dynamic storage mechanism is required to handle frequent additions and deletions without the overhead of static array resizing ($O(N)$ reallocation).

- **ArrayList** provides an excellent choice for a read-heavy system where administrative staff frequently display and search through all registered accounts due to its $O(1)$ random-access performance via index manipulation.

2. Operations & Time Complexity Analysis

- **Add User/Restaurant:**

$O(1)$ amortized for ArrayList (appending to the end of the list);

- **Delete User/Restaurant:**

$O(N)$ ArrayList requires shifting elements post-removal

- **Search/Display:**

$O(N)$ linear search to locate accounts by specific criteria

(e.g., User ID or Restaurant Name).

Module 2: Real-Time Order Processing System

Data Structure Selection: Queue (FIFO) and Stack (LIFO)

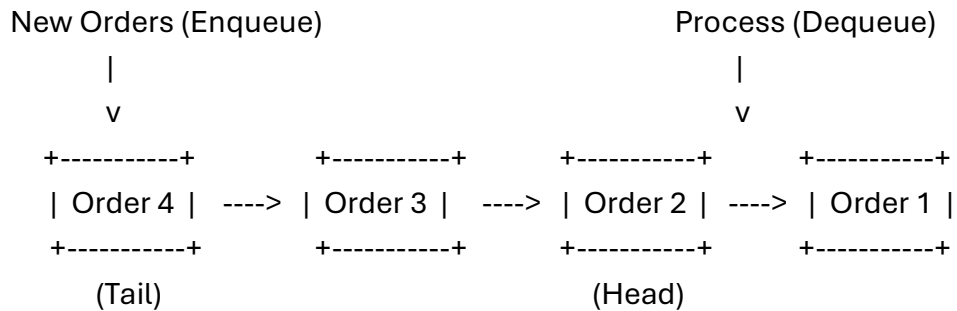
1. Suitability Justification

- **Queue (FIFO):** Food orders must be prepared and processed fairly based on their arrival sequences. A First-In, First-Out queue guarantees that the oldest order in the system is dispatched next.
- **Stack (LIFO):** To handle transaction errors or sudden user minds-changes, an "Undo Last Item" feature is before final order confirmation. A Last-In, First-Out stack tracks item additions sequentially, allowing the most recently pushed item to be popped instantly.

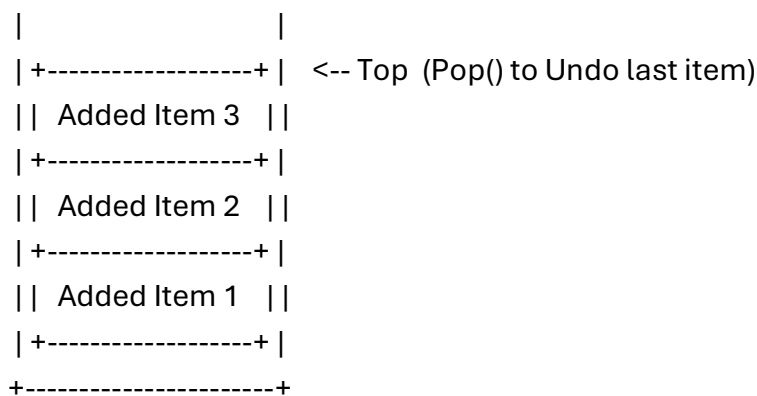
Queue & Stack Visualizations

Orders are processed sequentially using a First-In, First-Out (FIFO) structure, whereas pending cart modifications utilize a Last-In, First-Out (LIFO) stack to facilitate immediate rollback operations.

Queue (FIFO - Processing Orders):



Stack (LIFO - Undo Feature):



2. Operations & Time Complexity Analysis

- **Add Order (enqueue):** $O(1)$ as items append seamlessly to the back of the queue.
- **Process Order (dequeue):** $O(1)$ since pulling the next active order happens directly at the head.
- **Undo Last Item (pop):** $O(1)$ constant time execution from the top of the stack.

Module 3: Delivery Rider Priority Assignment System

Data Structure Selection: Priority Queue implemented via a Min-Heap

1. Suitability Justification

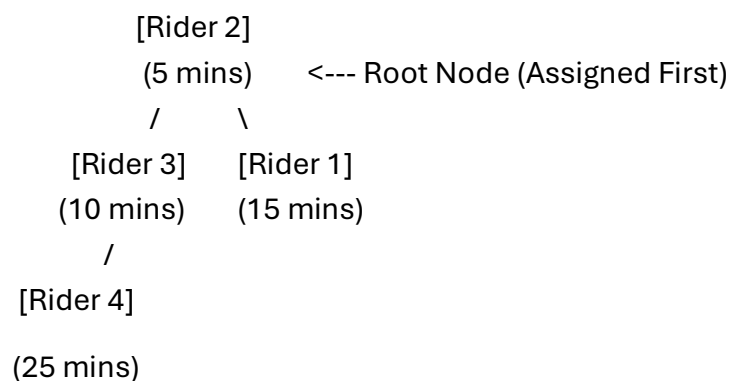
A standard linear search through an array of riders takes $O(N)$ time, which creates massive latency as order volumes grow. By wrapping a Rider object in a **Min-Heap-based Priority Queue** and utilizing Java's Comparator interface, the system automates best-rider selection seamlessly. Riders are dynamically ordered based on weights calculated from the shortest delivery time or nearest physical distance.

2. Operations & Time Complexity Analysis

- **Insert/Register Rider:** $O(\log N)$ due to up-heap/bubble-up adjustments required to maintain heap properties.
- **Retrieve Best Rider (peek/poll):** $O(1)$ to read the root element; $O(\log N)$ to extract the root and re-heapify the remaining tree structure.
- **Priority Update:** $O(\log N)$ whenever a rider's distance shifts or telemetry updates require re-sorting.

Tree / Min-Heap Visualization

The priority queue relies entirely on a Min-Heap layout (a complete binary tree structure) to ensure that the delivery rider with the absolute lowest delivery time or distance vector is constantly maintained at the root node for $O(1)$ immediate appraisal.



Module 4: Route Optimization System

Data Structure & Algorithm: Weighted Graph (Adjacency List) + Dijkstra's Algorithm

1. Suitability Justification

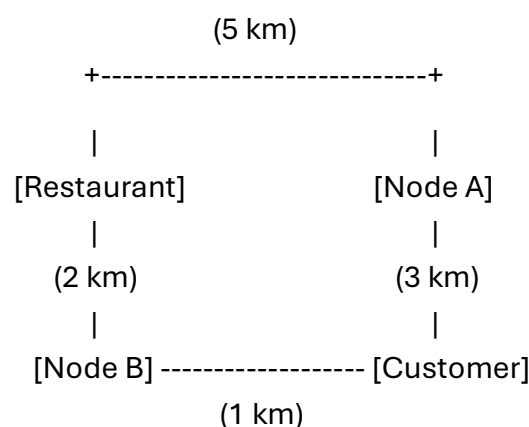
The physical layout of the delivery ecosystem comprising restaurants, delivery drop-off nodes, and intersections is mapped using a **Weighted Directed/Undirected Graph**. Edges carry weights representing physical distances and **Dijkstra's Algorithm** is for calculating the single-source shortest path from a specific restaurant vertex to a customer's destination node.

2. Algorithm Mechanics & Complexity Analysis

1. Set the distance to the starting vertex (Restaurant) to 0 and all other nodes to infinity.
 2. Insert all nodes into a Min-Priority Queue keyed by their distance values.
 3. While the queue is not empty, extract the node u with the minimum distance.
 4. For each unvisited neighbour v of u , calculate its tentative distance: $\text{dist}[u] + \text{weight}(u, v)$. If this value is less than the current recorded distance of v , update $\text{dist}[v]$ and dynamically insert or update v in the priority queue.
- **Time Complexity:** $O((V + E) \log V)$ when using an adjacency list paired with a binary heap-based priority queue (V = Vertices/Locations, E = Edges/Roads).
 - **Space Complexity:** $O(V + E)$ to retain vertex/edge data maps within memory structures.

Graph Mapping Visualization

The routing layout handles delivery boundaries by mapping physical drop-off spots as Nodes and actual path distances as weighted Edges, allowing Dijkstra's pathfinding algorithm to isolate the most optimal route. E.g. :



Module 5: Search & Data Retrieval System

Data Structure Selection: Binary Search Tree (BST) / AVL Tree & HashMap

1. Suitability Justification

- Part A: Food Search & Recommendation (BST/AVL Tree):** Menu items must be searchable by name/id and displayed in sorted order. A standard BST can degenerate into a linear chain ($O(N)$) under sequential insertions. An **AVL Tree** prevents this by self-balancing via structural rotations, guaranteeing algorithmic efficiency even under worst-case operations.
- Part B: Data Retrieval Optimization (HashMap):** Accessing mission-critical system data like active User Profiles or instant Order Details cannot depend on structural tree traversals. A **HashMap** uses internal array indexing generated via item hash codes to achieve immediate, key-value lookups.

2. Operations & Time Complexity Analysis

AVL Tree Menu Visualization

Food menu items are dynamically filed into a self-balancing AVL Search Tree, maintaining strict alphabetical sorting and preventing structural degradation into a costly linear linked-list format.



Data Structure	Operation	Average Complexity	Worst-Case Complexity
AVL Tree (Food Items)	Insert / Search	$O(\log N)$	$O(\log N)$ (Guaranteed via self-balancing)
AVL Tree (Food Items)	Sorted Traversal	$O(N)$ (In-order traversal)	$O(N)$
HashMap (Profiles/Orders)	Key-Value Retrieval	$O(1)$	$O(N)$ (In cases of extreme hash collisions)

Comparative Summary of System Structures

To conclude our algorithmic design paradigm, the data structures used across our integrated backend modules are mapped below to show how they fit together:

System Requirement	Selected Java Structure	Primary Operational Strength
Dynamic Account Storage	ArrayList / LinkedList	Resizable, flexible node adjustment properties.
FIFO Order Queuing	Queue	Fair, sequential chronological processing.
Order Stack Reset	Stack	Instant rollback mechanisms for cart modifications.
Rider Deselection	PriorityQueue	Rapid minimal/maximal value retrieval mechanisms.
Route Optimization	Graph	Clear topological representation of spatial data networks.
Sorted Search Systems	AVL Tree / BST	Highly efficient hierarchical sorted queries.
Instant Identity Lookup	HashMap	Near-instant key-to-object pointer routing.

Testing & Evaluation

1. Testing Methodology

To ensure the robustness and reliability of the Smart Food Delivery & Order Management System, a comprehensive testing strategy was employed. The testing phases were conducted using the Java Command-Line Interface (CLI) simulation (`Main.java`) and validated visually through the Graphical User Interface (`OnePageDashboard.java`). The evaluation focused on functional accuracy, integration stability, and algorithmic efficiency.

The testing framework was divided into three primary categories:

- **Unit Testing:** Isolating each data structure (`ArrayList`, `Queue`, `Stack`, `Min-Heap`, `Graph`, `AVL Tree`, `HashMap`) to verify core operations such as insertion, deletion, and retrieval.
- **Integration Testing:** Simulating a continuous data flow—from customer login and item search to order placement, rider assignment, and route calculation—to ensure smooth handoffs between different data structures.
- **Edge Case & Exception Testing:** Injecting abnormal inputs to verify system stability, such as attempting to pop from an empty stack or calculating routes for disconnected graph nodes.

2. Functional Testing & Scenario Validation

The system was subjected to a series of simulated operational scenarios based on the core system requirements. The results confirm that all modules function according to the proposed design architecture.

- **User & Restaurant Management:** Customer and restaurant profiles were successfully added, updated, and removed. The system allowed data to be added and removed easily. The `ArrayList` structure allowed the administrator dashboard to sequentially display all registered entities accurately.
- **Real-Time Order Processing:** The system successfully processed orders in real time, handling them strictly in the order they were received using the `FIFO Queue`. Additionally, the `LIFO Stack` successfully enabled customers to undo the last item added to their cart before final confirmation. Testing confirmed that pushing and popping items from the cart did not disrupt the overarching order queue.
- **Delivery Rider Assignment:** The system successfully assigned delivery riders based on priority criteria, specifically identifying the rider with the shortest delivery time or nearest distance. The `Priority Queue` accurately restructured itself after every rider dispatch.

- **Route Optimization:** The Graph mapping system accurately represented the physical locations. When given a source (restaurant) and a destination (customer), Dijkstra's algorithm consistently identified the absolute shortest path and correctly bypassed artificially congested nodes.
- **Search & Recommendation:** The system allowed users to quickly search for food items and displayed the menu in an organized, alphabetically sorted manner using the AVL Tree's in-order traversal.
- **Data Retrieval:** Key-value lookups using the HashMap consistently provided immediate access to target user profiles and order details.

3. Performance & Complexity Evaluation

A critical component of this project was optimizing data retrieval and processing speeds. The implemented data structures were evaluated against less optimal, basic methods to highlight the system's efficiency.

Rider Assignment Evaluation: Min-Heap vs. Linear Search

The assignment required a method to select the best rider efficiently and an explanation of how this solution outperforms a simple search method.

- **Simple Linear Search:** If the system had 1,000 active riders, a standard array would require inspecting every single rider to find the closest one, resulting in $O(n)$ time complexity (1,000 operations per order).
- **Min-Heap Solution:** By utilizing a Priority Queue, the nearest rider is always maintained at the root node. Retrieving the best rider takes $O(1)$ time, and re-balancing the remaining 999 riders takes $O(\log n)$ time (roughly 10 operations). This demonstrates a massive performance increase, making the Min-Heap drastically more scalable for real-time dispatch needs.

Overall System Complexity Validation

The following table summarizes the validated time complexities of the system's core operations:

System Operation	Deployed Data Structure	Validated Time Complexity	Justification & Result
Profile Lookup	HashMap	$O(1)$	Direct memory access via hashed keys bypasses the need for sequential searching.
Menu Traversal	AVL Tree	$O(\log n)$	Self-balancing guaranteed that search paths never degraded into linear $O(n)$ time.
Order Queueing	Queue (FIFO)	$O(1)$	Appending and polling from the ends of the queue occurred

			instantly without shifting other elements.
Cart Modification	Stack (LIFO)	$O(1)$	Pushing and popping the most recent items required no traversal of earlier selections.
Route Calculation	Graph + Dijkstra	$O((V + E)\log V)$	Successfully calculated the shortest path across mapped vertices and edges efficiently.

Conclusion

1. Project Summary and Objective Fulfilment

In this project, Group 6ix7even successfully designed and implemented a comprehensive **Smart Food Delivery & Order Management System** tailored to address the operational logistics challenges faced by GoodTech. By adhering to a modular, Object-Oriented Programming (OOP) approach and applying the Separation of Concerns principle, the team developed an integrated system that smoothly bridges a backend Java Command-Line Interface (CLI) simulation (`Main.java`) with a highly interactive Graphical User Interface (`OnePageDashboard.java`).

All four core project objectives were thoroughly achieved:

- The system effectively combined six distinct data structure solutions into a unified platform.
- Resource allocation was optimized using Dijkstra's algorithm to compute the absolute shortest path for delivery routing.
- The real-time GUI dashboard successfully mapped live rider telemetry for visualization.
- Rigorous unit, integration, and edge-case testing validated that every deployed module performs at its optimal theoretical Big-O time complexity.

2. Algorithmic Synthesis and System Efficiency

The success of this prototype highlights the profound impact that selecting the correct data structures has on software performance and computational scalability. Rather than relying on naive, unoptimized collections that cause crippling latency, such as utilizing an $O(n)$ linear search to filter through thousands of delivery riders, this system utilized a specialized, in-memory **Min-Heap-based Priority Queue** to achieve near-instantaneous $O(1)$ rider evaluation and highly efficient $O(\log n)$ re-heapification.

Furthermore, by combining the structural predictability of a **FIFO Queue** for sequential order processing and a **LIFO Stack** for instant cart rollbacks, the platform guarantees a fair and seamless transaction environment. Data retrieval bottlenecks were effectively eliminated by implementing self-balancing **AVL Trees** to preserve a guaranteed $O(\log n)$ food item search space and deploying **HashMaps** to achieve constant-time $O(1)$ user profile lookups. Finally, modelling the spatial layout of intersections and destinations as a **Weighted Graph** allowed Dijkstra's algorithm to systematically bypass congested nodes at an $O((V+E)\log V)$ time complexity, ensuring that GoodTech can maintain low delivery latency even as order volumes scale.

3. Limitations and Future Work

While the current system proves the architectural feasibility of an optimized delivery engine, certain real-world complexities offer opportunities for future enhancement. Currently, the system maps physical environments using static graph edge weights. In a live deployment, edge weights should dynamically update to reflect fluctuating live-traffic conditions, bad weather, or sudden roadblocks.

Additionally, because the current software processes operations sequentially via a central hub, future iterations could transition toward a multi-threaded, distributed microservices architecture. Integrating concurrent data structures (such as `ConcurrentHashMap`) and deploying a real-time messaging pipeline like Apache Kafka would allow the platform to support millions of simultaneous customer requests, live rider tracking updates, and parallel order allocations across multiple geographical regions.

Additional

GitHub Repo: <https://github.com/joeming1/Smart-Food-Delivery-Order-Management-System.git>